

Computer Vision for Automated Driving

Elias W.J. Müller

FS26

Contents

1	Part I: Fundamentals	2
1.1	Definitions	2
1.2	Optical Autonomous Driving Sensors	2
1.2.1	Light Detection and Ranging (Lidar)	3
1.2.2	Camera	5
1.2.3	Radar	5
1.2.4	Sensor Distribution on Car	6
2	Part II: Semantic Perception	7
2.1	Semantic Segmentation	7
2.1.1	Definition and Basics	7
2.1.2	Exemplar Driving Datasets	7
2.1.3	Convolutional Neural Networks Primer	7
2.1.4	Encoder–Decoder Paradigm	9
2.1.5	Fundamental Architectures and Modules	10
2.2	Object Detection	11
2.2.1	Fundamentals and Definitions	11
2.2.2	Two Stage R-CNN	12
2.2.3	Direct Detectors	14
2.3	Instance Segmentation	15
2.3.1	Mask R-CNN	15
2.4	Panoptic Segmentaton	16
2.4.1	Fundamentals and Definitions	16
2.4.2	Instance-semantic Framework for Panoptic Segmentation	16
2.4.3	Unified Mask-based Framework for Segmentation	17
3	Part III: Geometric Perception and Localization	19
3.1	Depth and 3D Estimation	19
3.1.1	Supervised Methods	19
3.1.2	Stereo Depth Estimation	20
3.1.3	Self-Supervised Monocular Depth Estimation from Stereo	22
3.1.4	Self-Supervised Depth and Motion Estimation from Video	22
3.2	3D Classification, Segmentation, and Detection	23
3.2.1	Foundations and Definitions	23
3.2.2	3D Classification and Semantic Segmentation on Point Clouds	23
3.2.3	3D Object Detection on Point Clouds	25
3.3	3D Reconstruction and Localization	27
4	Part IV: Robust Perception: Multi-Modal, Multi-Domain and Multi-Agent Methods	28
5	Part V: Temporal Perception	29

1 Part I: Fundamentals

1.1 Definitions

Definition 1 (Computer Vision). *Inverse algorithmic process to infer properties of the world based on sensory observations.*

Definition 2 (Autonomous Car (ADS-DV)). *ADS-equipped vehicle designed for driverless operation within its given operational design domain.*

Definition 3 (Automated Driving System (ADS)). *Hardware and software collectively capable of performing the entire DDT on a sustained basis, possibly limited to a specific ODD.*

Definition 4 (Operational Design Domain (ODD)). *Conditions (environmental, geographical, temporal, traffic/roadway) under which a given ADS is designed to function.*

Definition 5 (Dynamic Driving Task (DDT)). *All real-time operational and tactical functions for on-road driving (excludes strategic functions). Subtasks: (1) lateral control, (2) longitudinal control, (3) OEDR monitoring, (4) OEDR execution, (5) maneuver planning, (6) conspicuity enhancement.*

Definition 6 (OEDR). *Object and Event Detection and Response. DDT subtasks 3–4: monitor driving environment via detection, recognition, classification, response preparation, and response execution.*

Definition 7 (Driving Automation System). *Hardware and software capable of performing part or all of the DDT. $ADS \subseteq$ Driving Automation Systems. Four core modules: (1) semantic and geometric **Perception** (2) **Prediction** from temporal perception (3) path and motion **Planning** via response determination (4) **Control***

Level	Name	Narrative definition	Execution of steering and acceleration/deceleration	Monitoring of driving environment	Fallback performance of dynamic driving task	System capability (driving modes)
Human driver monitors the driving environment						
0	No Automation	the full-time performance by the human driver of all aspects of the dynamic driving task, even when enhanced by warning or intervention systems	Human driver	Human driver	Human driver	n/a
1	Driver Assistance	the driving mode-specific execution by a driver assistance system of either steering or acceleration/deceleration using information about the driving environment and with the expectation that the human driver perform all remaining aspects of the dynamic driving task	Human driver and system	Human driver	Human driver	Some driving modes
2	Partial Automation	the driving mode-specific execution by one or more driver assistance systems of both steering and acceleration/deceleration using information about the driving environment and with the expectation that the human driver perform all remaining aspects of the dynamic driving task	System	Human driver	Human driver	Some driving modes
Automated driving system ("system") monitors the driving environment						
3	Conditional Automation	the driving mode-specific performance by an automated driving system of all aspects of the dynamic driving task with the expectation that the human driver will respond appropriately to a request to intervene	System	System	Human driver	Some driving modes
4	High Automation	the driving mode-specific performance by an automated driving system of all aspects of the dynamic driving task, even if a human driver does not respond appropriately to a request to intervene	System	System	System	Some driving modes
5	Full Automation	the full-time performance by an automated driving system of all aspects of the dynamic driving task under all roadway and environmental conditions that can be managed by a human driver	System	System	System	All driving modes

Figure 1: SAE levels of driving automation

1.2 Optical Autonomous Driving Sensors

Countless sensor choices exist. We make the distinction of **external** (camera, lidar, radar, event camera, GPS, IMU, speedometer, ...) and **internal** (camera, thermal camera, microphone) sensors. Our focus further lies on the *Optical Sensors*.

Optical Sensors can be categorized along the following features:

- *Wavelength*: spectral band of light the sensor operates in (e.g. visible, near-infrared, millimeter-wave).
- *Active or Passive*: whether the sensor emits its own signal (active) or relies on external illumination (passive).
- *Resolution*: spatial density of measurements; determines fineness of detail captured.
- *Latency / Frame Rate*: temporal frequency at which full measurements are acquired.
- *Field of View* (FOV): angular extent of the observable scene, horizontal and vertical.
- *Range*: maximum distance at which the sensor can reliably detect objects.

1.2.1 Light Detection and Ranging (Lidar)

Characteristics: Active sensor, near-infrared wavelength (905nm), direct range measurements via time of flight (TOF), i.e., timing of light emission until recapture of photons, large horizontal and fair vertical FOV (3D), fair resolution

Pulsed Lidar

Multiple short laser pulses emitted in different directions and at different times. Each measurement: pulse emitter → beam steering → target → photodetector (APD). A timer measures the round-trip delay to infer range R_0

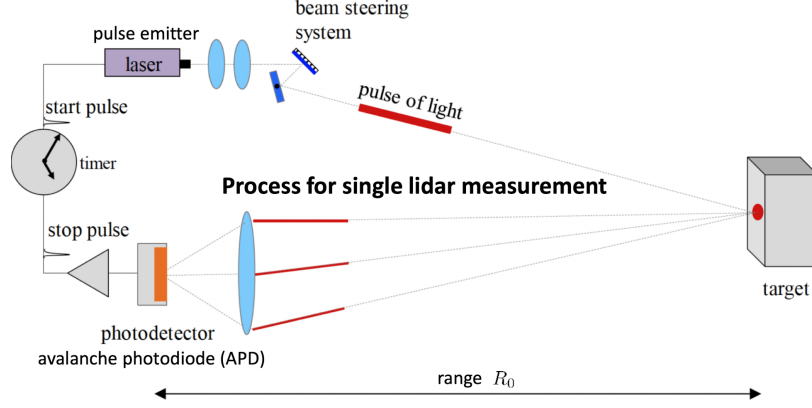


Figure 2: Lidar working principle

Beam Configurations.

Real beams diverge; receivers have finite FOV. Two configurations:

- *Coaxial*: transmitter and receiver share a common optical axis.
- *Bistatic*: optical axes of transmitter and receiver are parallel but offset. The **crossover function** $\xi(R) \in [0, 1]$ gives the fraction of the illuminated area observed by the receiver as a function of range. For bistatic: $\xi(R) = 0$ at short range (no crossover), $\xi(R) = 1$ beyond some R_2 (full crossover).

Pulse Waveform (Sinusoidal Form).

$$P_T(t) = \begin{cases} P_0 \sin^2\left(\frac{\pi}{2\tau_H} t\right), & 0 \leq t \leq 2\tau_H, \\ 0, & \text{otherwise.} \end{cases}$$

where $P_T(t)$: emitted radiant power, P_0 : peak power, τ_H : half-power pulse width (10–20 ns), $c\tau_H$: spatial half-power width (3–6 m).

Physically more realistic is the square-exponential form (left heavy, right tailed) – omitted here.

Linear System Model.

Received radiant power as a convolution of emitted power with the optical system impulse response:

$$P_R(R) = C_A \int_0^{2R/c} P_T(t) H\left(R - \frac{ct}{2}\right) dt$$

where $P_R(R)$: range-dependent received power, $H(R)$: impulse response of the optical system, C_A : system constant (independent of time and range). $ct/2$ maps time t to the corresponding range (round-trip).

Intuition: $H(R)$ is what we would receive from a perfectly concentrated (Dirac) pulse; the actual response is a smeared version due to finite pulse width - ambiguity from temporal profile/ early or late photon.

Impulse Response Decomposition.

$$H(R) = H_C(R) H_T(R)$$

- *Optical channel*: $H_C(R) = \frac{T^2(R)}{R^2} \xi(R)$, where $T(R) = \exp(-\alpha R)$ is the one-way transmission loss (α depends on visibility, assumes homogeneous scattering medium).
- *Target*: $H_T(R) = \rho_0 \delta(R - R_0)$ in clear weather, where ρ_0 is the differential reflectivity.

Clear Weather Simplification.

With $\alpha = 0$ (no attenuation – clear air) and $\xi(R_0) = 1$:

$$H(R) = \frac{1}{R_0^2} \rho_0 \delta(R - R_0)$$

Substituting into the linear system model yields:

$$P_R(R) = \begin{cases} \frac{C_A P_0 \rho_0}{R_0^2} \sin^2\left(\frac{\pi(R - R_0)}{c\tau_H}\right), & R_0 \leq R \leq R_0 + c\tau_H, \\ 0, & \text{otherwise.} \end{cases}$$

Intuition: The target acts as a Dirac delta in range, so the convolution merely shifts and scales the emitted waveform — the received pulse preserves the shape of P_T , centered at R_0 . The $1/R_0^2$ decay is the inverse-square law: emitted power spreads over a sphere of area $\propto R_0^2$. The proportionality to ρ_0 is why lidar returns per-point *intensity* alongside range. The pulse occupies a range interval of width $c\tau_H$, so shorter pulses yield finer range resolution at the cost of less energy per pulse.

Features of Pulsed Lidar.

- *Ranging precision*: limited by temporal resolution of timing electronics. Typical $\sim 0.1 \text{ ns} \Rightarrow \sim 1.5 \text{ cm}$.
- *Maximum range*: limited by signal to noise ratio (SNR) (attenuation + detector noise/jitter): 100–200 m.
- *Range-rate trade-off*: maximum unambiguous range is inversely proportional to pulse repetition rate (aliasing) – upper limit of the integral

Acquisition of 3D Point Clouds.

Each pulse yields a single range measurement along a direction (α, ϵ) . To obtain a 3D point cloud, the sensor scans across a range of azimuth α and elevation ϵ values, which jointly define the FOV. Spherical-to-Cartesian conversion:

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = \mathbf{h}^{-1}(r, \alpha, \epsilon) = \begin{bmatrix} r \cos \alpha \cos \epsilon \\ r \sin \alpha \cos \epsilon \\ r \sin \epsilon \end{bmatrix}$$

Each point additionally carries an *intensity* value $= f(P_R(R_0))$, derived from the peak received power.

Scanning Mechanisms.

Taxonomy: non-scanning (flash) vs. scanning lidar. Scanning subdivides into non-mechanical (OPA, solid-state) and mechanical (MEMS, motorized optomechanical). Driving-relevant types are motorized mechanical and MEMS scanners.

Motorized mechanical: nodding-mirror tilt controls vertical FOV; rotating housing ($> 1000 \text{ RPM}$) provides panoramic 360° horizontal FOV. Modern systems use laser/photodiode arrays $\Rightarrow$ mechanical rotation only along azimuth. Horizontal angular resolution up to 0.1° , max range up to 200 m.

MEMS: tiny oscillating mirrors (few mm diameter) steer individual pulses. 1D MEMS scans one angular dimension (needs laser array or motor for the other); 2D MEMS scans both. Resonant scanning: frequency = resonant frequency of mirror, high frame rate, sensitive to environmental variation. Non-resonant: more robust, lower frame rate.

Motorized Mechanical	MEMS
Panoramic 360° horizontal FOV	Limited ($\sim 120^\circ$) horizontal FOV
Uniform azimuthal scanning pattern	Complex / less interpretable pattern
Bulky, vulnerable to wear	Small, no mechanical wear
Imbalanced vert. vs. horiz. resolution	Balanced resolution
Low frame rate (10–20 Hz)	Slightly higher frame rate
Expensive	Less expensive

Measurement Errors in Lidar.

- *Sensor*: timing jitter in arrival detection; rotary encoder uncertainty (angular error $\rightarrow$ distortion).
- *Scene*: range ambiguity (multiple waveform maxima); non-Lambertian surfaces (absorptive $\rightarrow$ no return, specular $\rightarrow$ multi-path); motion distortion from finite frame rate (not rectifiable, unlike ego-motion).
- *Optical medium*: attenuation (e.g. fog $\rightarrow$ reduced peak amplitude); backscattering (e.g. snow/rain $\rightarrow$ false peaks, spurious points).
- *Light sources*: ambient NIR interference (minor); cross-talk from other lidars.

1.2.2 Camera

Characteristics: Passive sensor, visible spectrum (typically three sensitivity profiles / channels), very high resolution, fair frame rate, moderate FOV for perspective cameras, cheap.

Pinhole Camera Model.

Ideal pinhole = center of projection $\mathbf{O}_c$. All scene rays pass through a single point onto the image plane. Virtual image plane (in front of pinhole): image is not inverted. This yields a *perspective projection*.

Homogeneous Coordinates.

Unify representation of points and planes ($ax + by + cz - d = 0$) by adding one dimension:

$$\mathbf{P} = \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} \quad (\text{point}), \quad \mathbf{\Pi} = \begin{pmatrix} a \\ b \\ c \\ -d \end{pmatrix} \quad (\text{plane}),$$

where $\mathbf{\Pi} \cdot \mathbf{P} = 0$ iff $\mathbf{P}$ lies on $\mathbf{\Pi}$.

Geometric Transformations.

Notation: ${}^F\mathbf{P}$ denotes coordinates of point $\mathbf{P}$ in frame F .

- *Translation:* ${}^B\mathbf{P} = {}^A\mathbf{P} + {}^B\mathbf{O}_A$, where ${}^B\mathbf{O}_A$ is the origin of frame A expressed in frame B .
- *Rotation:* ${}^B\mathbf{P} = {}^B\mathbf{R} {}^A\mathbf{P}$, where ${}^B\mathbf{R} = ({}^B\mathbf{i}_A \ {}^B\mathbf{j}_A \ {}^B\mathbf{k}_A)$ and $\mathbf{R}^T\mathbf{R} = \mathbf{I}$.
- *Rigid / Euclidean:* ${}^B\mathbf{P} = {}^B\mathbf{R} {}^A\mathbf{P} + {}^B\mathbf{O}_A$. In homogeneous coordinates:

$$\begin{bmatrix} {}^B\mathbf{P} \\ 1 \end{bmatrix} = \begin{bmatrix} {}^B\mathbf{R} & {}^B\mathbf{O}_A \\ \mathbf{0}^T & 1 \end{bmatrix} \begin{bmatrix} {}^A\mathbf{P} \\ 1 \end{bmatrix}$$

3D to 2D Perspective Projection.

Given a 3D point $\mathbf{P}_c = (x_c, y_c, z_c)^T$ in camera coordinates, projection to 2D homogeneous pixel coordinates $\tilde{\mathbf{x}}_s$:

$$\tilde{\mathbf{x}}_s \sim \mathbf{K} \mathbf{P}_c, \quad \mathbf{K} = \begin{bmatrix} f & s & c_x \\ 0 & f & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

where f : focal length, (c_x, c_y) : principal point, $\mathbf{K}$ encoding the camera intrinsics (5DOF)

Intuition: Derived from similar triangles starting with: $-(x_s - c_x)/x_c = f/(-z_c)$.

Incorporating Extrinsics. rigid transformation from world frame to camera frame: $\mathbf{P}_c = [\mathbf{R} \mid \mathbf{t}] \tilde{\mathbf{P}}_w$

Complete Projection.

from 3D homogeneous world coordinates to 2D homogeneous image coordinates:

$$\tilde{\mathbf{x}}_s \sim \underbrace{\mathbf{K} [\mathbf{R} \mid \mathbf{t}]}_{\mathbf{C}} \tilde{\mathbf{P}}_w \quad \mathbf{C}: 3 \times 4 \text{ camera matrix (11 DOF: 5 intrinsics + 6 extrinsics)}$$

Photometric Image Formation.

Digital imaging pipeline (scene radiance $\rightarrow$ optics $\rightarrow$ sensor $\rightarrow$ ISP $\rightarrow$ output), BRDF-based (Bidirectional Reflectance Distribution Function) rendering equation, Lambertian vs. specular reflection. (details omitted)

1.2.3 Radar

Characteristics: Active sensor, 70–80 GHz (wavelengths $\sim$ few mm), measures *range and velocity*. Robust to fog/rain/snow (wavelength $\gg$ particle size $\Rightarrow$ no scattering). Large horizontal FOV, very narrow vertical FOV. Low azimuthal resolution ($\approx$ measurements for 360 deg), high range resolution. Low frame rate. Range: 200–600 m.

FMCW Scanning Radar.

Two waveform types: fixed-frequency pulses and frequency-modulated continuous wave (FMCW). Output is discretized into M azimuths $\times$ N range bins; per-bin measurement is received power.

Radar equation: $P_{\text{received}} = \frac{P_{\text{transmitted}} G \sigma A_e}{16\pi^2 r^4 L}$ where G : transmitter gain, σ : radar cross section (RCS, depends on target area/orientation/material), A_e : effective antenna aperture, L : loss factor, r : range. Note $1/r^4$ decay (vs. $1/R_0^2$ for lidar) because radar uses separate opening angles for transmitter and receiver. This is the radar analogue of the pulsed lidar received-power model: it determines how SNR degrades with range and target properties, and thus the sensor's effective detection limits.

FMCW Range and Velocity Measurement:

Modulation pattern: frequency of transmitted signal as a function of time.

Intermediate frequency (IF): difference between transmitted and received signal frequencies.

For a *static target*, a single linear frequency sweep (chirp, i.e. frequency ramping at constant rate df/dt) yields range directly from the IF: $r = \frac{c \Delta f}{2(df/dt)}$ where Δf is the IF.

For a *moving target*, there is ambiguity between TOF shift and Doppler shift Δf_v . Resolved using repeating triangular modulation pattern (up-chirp IF = Δf_1 , down-chirp IF = Δf_2):

$$\Delta f_v = \frac{\Delta f_2 - \Delta f_1}{2}, \quad \Delta f_r = \frac{\Delta f_1 + \Delta f_2}{2} \quad (1)$$

Δf_r gives range; Δf_v gives radial velocity relative to sensor.

Range-FFT: FFT of the IF signal yields the power-range spectrum for each azimuth. Each frequency component in the IF corresponds to a specific range, so the FFT separates targets at different distances. Real targets are not point-like $\Rightarrow$ energy spreads across multiple range bins, unlike the ideal single-peak response.

1.2.4 Sensor Distribution on Car

Typical ADS platform uses 20–40 sensors. Sensor placement affects covered FOV, cost, reliability, dirt management, serviceability, and crash survivability. Roof-mounted setups (common for robotaxis / data collection) are not acceptable for commercial vehicles. Changes in sensor 6D pose introduce dataset bias, potentially requiring full re-collection. Sensor suite composition is still under active definition in the industry.

2 Part II: Semantic Perception

2.1 Semantic Segmentation

2.1.1 Definition and Basics

Definition 8 (Semantic Image Segmentation). *Given a predefined class set $\mathcal{C} = \{1, \dots, C\}$, assign every pixel $\mathbf{x}_i$ of an input image $\mathbf{I} \in \mathbb{R}^{H \times W \times 3}$ a single class label $y_i \in \mathcal{C}$. No differentiation between distinct instances of the same class (cf. instance/panoptic segmentation).*

Data Format.

Supervised setup: dataset $\mathcal{D} = \{(\mathbf{I}^{(n)}, \mathbf{Y}^{(n)})\}_{n=1}^N$.

- *Input*: RGB image $\mathbf{I} \in \mathbb{R}^{H \times W \times 3}$.
- *Output / ground truth*: one-hot tensor $\mathbf{Y} \in \{0, 1\}^{H \times W \times C}$ with $\sum_{c=1}^C Y_{h,w,c} = 1$ for every pixel (h, w) . One channel per class.

Indexing convention: $h \in \{1, \dots, H\}$ row, $w \in \{1, \dots, W\}$ column, $c \in \{1, \dots, C\}$ class channel, k used as a dummy class index in sums.

Prediction Format.

Model outputs a logits tensor $\mathbf{Z} \in \mathbb{R}^{H \times W \times C}$. Per-pixel softmax gives the soft prediction tensor $\hat{\mathbf{Y}} \in [0, 1]^{H \times W \times C}$:

$$\hat{Y}_{h,w,c} = \frac{\exp(Z_{h,w,c})}{\sum_{k=1}^C \exp(Z_{h,w,k})}, \quad \hat{\mathbf{Y}}_{h,w,:} \in \Delta^{C-1}, \quad \text{Hard prediction: } \hat{y}_{h,w} = \arg \max_c \hat{Y}_{h,w,c}.$$

Categorical Cross-Entropy Loss.

$$\mathcal{L}(\mathbf{I}) = -\frac{1}{HW} \sum_{h=1}^H \sum_{w=1}^W \sum_{c=1}^C Y_{h,w,c} \log \hat{Y}_{h,w,c}.$$

Intuition: One-hot $\mathbf{Y}$ collapses inner c -sum to the single true-class term ($\log \hat{Y}_{h,w, y_{h,w}}$) $\rightarrow$ average over HW pixels.

Evaluation Metrics.

- *Pixel accuracy*: fraction of correctly classified pixels. Skewed by frequent classes (sky, road).
- *IoU (class c)*: $\text{IoU}_c = \frac{|\hat{P}_c \cap P_c|}{|\hat{P}_c \cup P_c|}$, with $\hat{P}_c, P_c$ the predicted/GT pixel sets.
- *mIoU*: $\frac{1}{C} \sum_c \text{IoU}_c$. More stringent than pixel accuracy; mean over classes prevents large object bias (a wrong prediction will be wrong in all instances).

2.1.2 Exemplar Driving Datasets

Dataset	Scale / resolution	Notes
Cityscapes	5k fine + 20k coarse, 1024×2048	30 classes / 8 categories; SOTA mIoU > 85%
Mapillary Vistas	$4000^+ \times 3000^+$	66 classes (37 instance-level); global, multi-viewpoint
Foggy Cityscapes	inherits Cityscapes labels	fog simulation at varying densities
GTA5	24,966 frames, HR	label propagation via rendering pipeline (~ 7 s/img)
ACDC	4006 adverse-cond. images	fog/night/rain/snow + clear correspondences
ApolloScape	140k pixel-level annot.	dense traffic
IDD	unstructured environments	unconventional classes, high intra-class diversity
WildDash	evaluation/testing	organized by visual hazards

2.1.3 Convolutional Neural Networks Primer

Limitations of MLPs on Images.

Vectorizing an $H \times W$ image as MLP input

1. destroys 2D structure and local correlations; spatial invariances must be relearned;
2. explodes the parameter count (e.g. 400×250 image, 10k hidden $\Rightarrow \sim 10^9$ parameters in the first layer alone).

Discrete 2D Convolution.

Slide the kernel $\mathbf{K} \in \mathbb{R}^{m \times n}$ over input $\mathbf{X}$; at each location multiply the overlapping pixel-weight pairs and sum:

$$(\mathbf{X} * \mathbf{K})[p, q] = \sum_{i=1}^m \sum_{j=1}^n X[p+i-1, q+j-1] K[i, j].$$

Formally a *cross-correlation*; it differs from convolution proper by a kernel flip.

Intuition: The operation is *local*, *linear*, and *shift-equivariant* (commutes with translation). The same parameters are used across all spatial locations – *parameter sharing*. The 2D structure of the input is preserved in the output.

Convolutional Layer: General Formulation.

At layer ℓ , the pre-activation tensor of the previous layer

$$\mathbf{z}_{\ell-1} \in \mathbb{R}^{H_{\ell-1} \times W_{\ell-1} \times d_{\ell-1}}$$

is convolved (after element-wise activation g) with d_ℓ 3D kernels

$$\mathbf{W}^{(\ell, c)} \in \mathbb{R}^{m_\ell \times n_\ell \times d_{\ell-1}}, \quad c = 1, \dots, d_\ell,$$

each paired with a scalar bias $b_c^{(\ell)}$. One distinct kernel channel exists for every input channel; the kernel is therefore a 3D tensor whose depth matches $d_{\ell-1}$.

Single Convolutional Neuron.

For output channel c at spatial location (p, q) , the pre-activation of layer ℓ is the sum of 2D cross-correlations against the activated outputs of the previous layer, plus the bias:

$$z_{\ell, c}[p, q] = \sum_{k=1}^{d_{\ell-1}} \sum_{i=1}^{m_\ell} \sum_{j=1}^{n_\ell} w_{k, c}^{(\ell)}[i, j] g(z_{\ell-1, k}[p+i-1, q+j-1]) + b_c^{(\ell)},$$

where $g(\cdot)$ is the (non-linear) activation function and $d_{\ell-1}$ denotes the number of input channels (i.e. the output channels of layer $\ell-1$, a hyperparameter of the preceding layer); m_ℓ, n_ℓ are the kernel's spatial dimensions.

Intuition: The 2D cross-correlation generalizes to multi-channel inputs as a triple sum (two spatial + one channel dim); the 3D kernel “stacks” the channel-wise correlations and the bias offsets the result. Parameters are shared across all spatial neurons of the same output channel.

Element-wise Nonlinearity.

The activation g is applied independently to every scalar entry of the pre-activation tensor $\mathbf{z}_\ell$; the activated value

$$g(z_{\ell, c}[p, q])$$

is what enters the next layer's convolution. The conv-and-bias step and the nonlinearity are conceptually separate sub-layers; g couples neither spatial positions nor channels.

Output Tensor.

Stacking the d_ℓ feature maps gives the 3D pre-activation tensor

$$\mathbf{z}_\ell \in \mathbb{R}^{H_\ell \times W_\ell \times d_\ell},$$

i.e. multiple features per spatial location, one per output channel. d_ℓ is a layer hyperparameter

Parameter Count.

Per output channel: $m_\ell n_\ell d_{\ell-1}$ kernel weights + 1 bias. Total parameters in layer ℓ :

$$\#\theta^{(\ell)} = d_\ell (m_\ell n_\ell d_{\ell-1} + 1).$$

Intuition: Cost scales with kernel footprint and the product of input/output channels, *not* with the spatial size of the input – this is what makes convolutional layers tractable on high-resolution images.

Pooling Layer.

Non-parametric, (sub)differentiable downsampling. Hyperparameters: window size, stride. Max pooling provides robustness to local input variations and enables higher feature dimensionality without spatial-memory explosion.

Practical Variations.

- *Padding*: valid convolution shrinks spatial dims. *Same* mode pads by $\lfloor m/2 \rfloor$ (odd m) to preserve them. Modes: zero, replicate, reflect.
- *Strided convolution*: kernel slides with step $s > 1$. Motivation: faster receptive-field growth, spatial down-sampling integrated into the conv. Often in early layers but pooling more common for downsampling. Higher stride $\rightarrow$ faster increase of receptive field size

Output spatial size (per dim):

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2P - m}{s} \right\rfloor + 1,$$

where H_{in} : input size, m : kernel size (per dim), P : padding width per border, s : stride.

Receptive Field.

Region of the input on which a neuron's output depends. Grows with depth, kernel size, and stride. Deep CNN layers eventually attain global receptive fields covering the entire image.

CNN for Image Classification.

Architecture: $(\text{conv} + \text{activation} + \text{pool})^L \rightarrow \text{flatten} \rightarrow \text{FC layers} \rightarrow \text{softmax}$. Final FC layers learn global interdependencies needed for the image-level prediction.

2.1.4 Encoder–Decoder Paradigm

Naive: Sliding-Window Classification.

Extract a patch around each pixel, classify its center with a CNN.

Problems: massive redundant computation; no global context (esp. for large objects). Does not scale to HR images.

End-to-End Architectures.

Single forward pass from $\mathbf{I}$ to full label map $\hat{\mathbf{Y}}$. Goal: share computation across all spatial locations. Strategy: adapt classification CNNs to dense prediction.

Three Challenges.

(I) *Reduced feature resolution* from repeated striding/pooling, (II) *Multi-scale objects*, (III) *Reduced localization accuracy* from CNN invariance

Fully Convolutional Networks (FCN).

Replace the FC layers of a classification network with convolutional layers $\Rightarrow$ coarse high-level feature maps at low resolution. Per-pixel prediction at original image resolution then requires upsampling.

Naive Resolution Recovery.

Remove pooling/striding entirely. Requires larger kernels for the same receptive field; every feature map $\mathbf{z}_\ell$ stays at HW instead of shrinking geometrically, so activation memory $\sum_\ell H_\ell W_\ell d_\ell = HW \sum_\ell d_\ell$ scales linearly with depth (no geometric decay), and the same factor inflates per-layer FLOPs $\Rightarrow$ heavy compute/memory cost and overfitting risk.

Encoder–Decoder.

- *Encoder*: maps $\mathbf{I}$ to a low-resolution, high-dimensional feature tensor; aggregates context.
- *Decoder*: upsamples encoder features back to image resolution while reducing feature dimensionality.
- Spatial resolution contracts inside the encoder (high $\rightarrow$ low) and is recovered by the decoder (low $\rightarrow$ high); feature dimensionality follows the opposite trajectory.

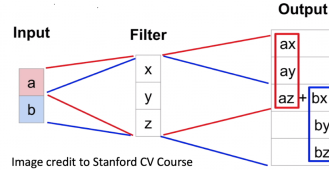
In-Network Upsampling.

Unpooling.

During encoder max-pooling, store argmax positions in *switch variables*; in the decoder, place each activation back at its recorded location. Produces an enlarged but *sparse* feature map. Not learnable + symmetric architecture for corresponding pairs.

Transpose Convolution (Deconvolution).

Learnable upsampling for dense feature maps. Each input scalar weights a copy of the kernel; the weighted kernel is placed into the output at a location determined by the input position and the stride. Where output positions overlap across different input pixels, the contributions are summed. (Crop might be required to get exact target size)

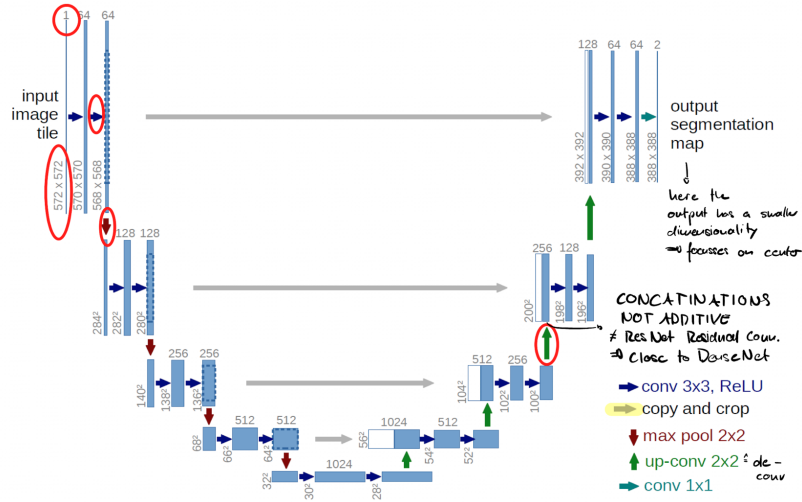


2.1.5 Fundamental Architectures and Modules

U-Net.

Symmetric encoder-decoder with *skip connections*: at each decoder stage, high-resolution encoder features are concatenated with upsampled decoder features along the channel dim, addressing the resolution challenge with fine grained localization information, followed by convolutions.

- Contracting path: $\downarrow$ “Where”, $\uparrow$ “What” (context).
- Expansive path: transpose convolutions + concatenations recover spatial detail.



Atrous (Dilated) Convolution.

The *effective* receptive field is smaller than the formal one: border connections are sparse. Atrous convolution inserts holes between kernel taps. For kernel $\mathbf{K} \in \mathbb{R}^{m \times n}$ and rate r :

$$(\mathbf{X} *_r \mathbf{K})[p, q] = \sum_{i=1}^m \sum_{j=1}^n X[p + ri, q + rj] K[i, j].$$

Standard convolution corresponds to $r = 1$. Stacking atrous convolutions with growing r yields *exponential* expansion of receptive field size without resolution loss or coverage gaps and fixed number of weights. We use 0-inserts to retain tensor calculations.

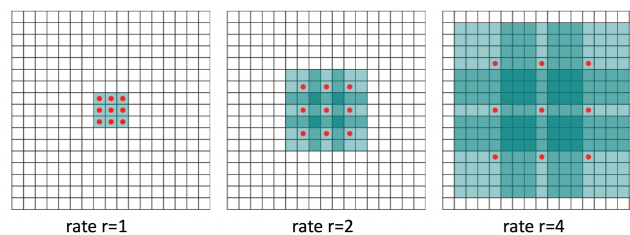


Figure 3: Atrous convolution (red dots: kernel taps)

Stride vs Atrous.

Stride downsamples the output to grow receptive field; atrous spreads the kernel to grow receptive field without downsampling. That's why atrous is preferred for dense prediction (segmentation) — you keep spatial detail.

Attention.

Provides a *variable visual receptive field*. For tokens $\mathbf{X} \in \mathbb{R}^{N \times d}$, learned projections $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$ produce

$$\mathbf{Q} = \mathbf{XW}_Q, \quad \mathbf{K} = \mathbf{XW}_K, \quad \mathbf{V} = \mathbf{XW}_V.$$

- *Query*: representation of the token used to compare it with other tokens.
- *Key*: reference representation by which other tokens are compared against the token.
- *Value*: output representation contributed by the token when it gets “indexed”.

Scaled dot-product attention:

$$\text{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}}\right) \mathbf{V}.$$

Softmax taken along rows (per-query distribution). Division by $\sqrt{d_k}$ keeps gradient magnitudes well-scaled. Complexity: $\mathcal{O}(N^2d)$ – quadratic in tokens.

Intuition: Query/key match measures relevance of every other token; the row-softmax weights aggregate values $\Rightarrow$ each token is a learned linear mixture of all others. Globally connected receptive field at every layer.

Transformer Block.

Multi-head self-attention (concatenate h parallel attention heads $\rightarrow$ linear projection) followed by a position-wise MLP. Residual connections and layer normalization around each sub-layer.

Transformers for Dense Prediction (Computer Vision).

ViT-style large patches lose spatial precision $\Rightarrow$ poor for segmentation. Naive fix: restrict attention to local windows of $M \times M$ patches. Complexity becomes *linear* in the number of patches (for fixed M). Drawback: receptive field bounded by the window across all layers.

Swin Transformer.

Shifted-window attention: alternate between W-MSA (window-based multi-head self-attention) and SW-MSA (shifted-window MSA). The window grid in SW-MSA is offset relative to W-MSA, so a query token's receptive field at layer $\ell + 1$ is the union of layer- ℓ windows that overlap with the token's current local window.

Intuition: With enough shifts, local windowed attention reaches large receptive fields, keeping per-layer cost linear.

Hierarchical Structure.

Patch merging concatenates features of each 2×2 neighboring patch group and applies a linear projection: tokens reduce by $4 \times$ ($2 \times$ spatial downsampling), output dim set to $2d$. Yields a multi-scale feature pyramid; Swin can serve as a drop-in CNN-backbone replacement for the encoder of standard segmentation networks.

2.2 Object Detection

2.2.1 Fundamentals and Definitions

Definition 9 (Object Detection.). *Complement to semantic segmentation focussed on instance specific detection using bounding boxes. In essence the question of what?, where?*

Evaluation.

- **Precision:** accuracy of predictions (how many correct?): $\frac{TP}{TP+FP}$
- **Recall:** detection of true objects: $\frac{TP}{TP+FN}$
- **IoU:** overlap of two bounding boxes (i.e., predicted vs ground truth)

- **(m)AP:** The **Average Precision** is the **Area Under the Curve** (Precision-Recall). Core idea is to take ranked detection by confidence level and some IoU threshold (when does a prediction count as correct). Then the **Precision-Recall Curve** is given by these ranked predictions. Since Prec and $\text{Rec} \in [0, 1]$ the unit integral of AP is $\in [0, 1]$. We ensure monotonic decrease by replacing each precision value with the maximum precision value to the right of that recall level.
- **Interpolated AP:** use an n -point interpolated AP, and then use the mean-AP of these n points.

AP Note: Generally the AP is determined for a given class and iou-threshold. Generally, we apply a number of predefined thresholds (loose to tight) and average them for a given class. All per class APs are average at the end for the full AP $\rightarrow$ *average (class), average (IoU thresholds), average (precision @ different recall levels) precision.*

2.2.2 Two Stage R-CNN

Region-based CNNs use per-region computation for the detection task, rather than considering the full input in one loss function target. So the classifier only ever sees some number of potentially high quality regions, not the full $H_f W_f$ anchor grid.

Note: One-Stage detectors, have no separate proposal step. Every anchor is directly classified into $K + 1$ classes and regressed in a single forward pass (no proposal, every anchor, 1 box regression, faster, but lower quality).

R-CNN

1. **RPN:** off-the-shelf region proposal network (trained disjoint from this architecture).
2. **Crop & Warp** the patches for fixed network input size
3. **Forward** through network for feature map $\rightarrow$ VERY SLOW per-region pass required (≈ 2000)
4. **Linear Object Classifier** plus a parallel **Box Regressor** for a refined proposal localization

Fast R-CNN

1. **FCN transformation** of input into featurized repr. $f_I = f(I)$
2. unchanged RPN $r(I) \rightarrow$ Region Proposals have VERY POOR RECALL and are slow
3. **RoI Pooling** for a featurized repr of each proposal $g_i = g(f_I, r_i)$: project proposal to feature-map coords (divide by FCN stride s), snap fractional coords to integer feature-map cells, split into fixed $k \times k$ grid, max-pool per bin $\rightarrow$ fixed $k \times k \times C$ output regardless of proposal size.
4. **ConvNet/MLP** for preprocessing per proposal feature (input already has lower spatial resolution since we use feature maps) $\rightarrow h(g_i)$
5. **Multiple Heads** can be attached to these feature maps for task-specific predictions.

(Multi-Task) Loss

Since we have two sibling output layers we need to combine their losses for joint training,

- $L_{cls}(p, u) = -\log p_u$ as discrete probability dist per RoI over $K+1$ categories (CE-loss)
- $L_{loc}(t^u, v) = \sum_{i \in \{x, y, w, h\}} \text{smooth}_{L_1}(t_i^u - v_i)$ summing per-coord robust offsets, with $\text{smooth}_{L_1}(x) = 0.5x^2$ if $|x| < 1$ else $|x| - 0.5$ (Huber: L_2 near 0, L_1 in tails $\rightarrow$ less outlier-sensitive). Predicts $t^k = (t_x^k, t_y^k, t_w^k, t_h^k)$ for each of the K categories.
- **Combined:** each RoI is given a ground-truth class u and a BB-regression target v . The BB-loss is activated for all non-background classes ($\lambda[u \geq 1]$)

$$L(p, u, t^u, v) = L_{cls}(p, u) + \lambda[u \geq 1]L_{loc}(t^u, v)$$

Faster R-CNN

1. unchanged FCN transformation of input into featurized repr. $f_I = f(I)$
2. **Learned Region Proposals** using the already transformed f_I (computation is shared) $\rightarrow RPN(f_I)$:
A 3×3 *sliding window* scans f_I ; at each position it reads a feature vector that is fed to two sibling heads. At the same position we place k predefined *anchor boxes* (cartesian product of scales $\times$ ratios, e.g. $3 \times 3 = 9$).
Crucially the two boxes are decoupled: the 3×3 window only defines *where features are read from*; the anchor is the *reference frame for the predictions*. The heads output, *per anchor*:

- **objectness** $\in [0, 1]$: does *this anchor* (not the sliding window) contain an object?
- **box regression** (dx, dy, dh, dw) : offsets *relative to the anchor*, not the sliding window.

So a single feature vector yields k proposals at different scales/ratios sharing one center – the anchor sets the prior, the deltas fine-tune. Per-location output: $2k$ scores + $4k$ regression values $\rightarrow$ up to k co-located objects detectable per sliding position. Total $H_f W_f k$ proposals, filtered by objectness threshold + NMS before being passed as RoIs.

3. unchanged RoI Pooling, unchanged ConvNet/MLP, unchanged Multiple Heads

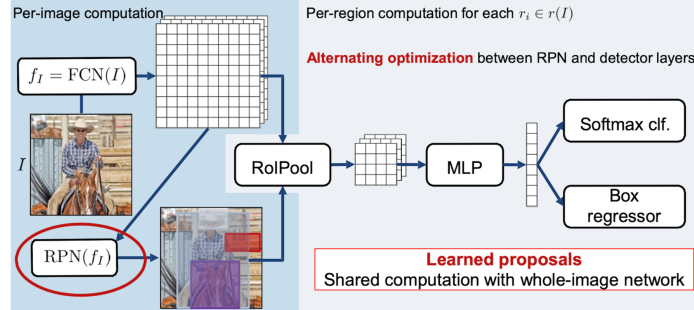


Figure 4: Faster R-CNN: per-image FCN backbone is shared between the RPN (learned proposals) and the per-region head (RoIPool $\rightarrow$ MLP $\rightarrow$ softmax + box regressor). Training alternates between RPN and detector layers.

RPN Loss & Anchor Assignment

Each anchor i gets a label p_i^* :

- **positive** ($p_i^* = 1$): IoU > 0.7 with any GT box (or the highest-IoU anchor for a GT, to guarantee a match)
- **negative** ($p_i^* = 0$): IoU < 0.3 with all GT boxes
- **ignored**: in between – no loss contribution.

One GT can match *multiple* anchors. Multi-task loss combines objectness CE and box-regression smooth- L_1 :

$$\mathcal{L}(\{p_i\}, \{t_i\}) = \frac{1}{N_{\text{cls}}} \sum_i \mathcal{L}_{\text{cls}}(p_i, p_i^*) + \lambda \frac{1}{N_{\text{reg}}} \sum_i p_i^* \mathcal{L}_{\text{reg}}(t_i, t_i^*)$$

$\mathcal{L}_{\text{cls}}$ runs over *all* pos+neg anchors; $\mathcal{L}_{\text{reg}}$ is gated by p_i^* – only positives get a regression loss (negatives have no GT box to regress to). Architecturally the sliding window feeds a 256-d intermediate vector that branches into the $2k$ -d cls head and $4k$ -d reg head.

Faster R-CNN with Feature Pyramid Network (FPN)

Rather than using a same scale feature map, we include a multi-scale feature pyramid, i.e., f_I is no longer just one repr at one scale. This improves *Scale Invariance* and *Localization Equivariance*.

Different Strategies:

- **Image Pyramid**: At each scale predict individually - standard, easy but SLOW
- **Multiscale Features (Single-scale Map)**: single feature map by downsampling - fast but suboptimal especially for small objects
- **Naive In-Network Pyramid**: use intermediate features while downsampling for predictions - fast, suboptimal
- **Feature Pyramid Network**: enrich high-res features from low-res features in a top-down manner - fast, less suboptimal, since we use the strong features of low resolution with the fine grained nature of the high resolution

Issues with Indirect Detectors

Anchor-based methods are *indirect*: predictions are made w.r.t. a hand-designed anchor grid rather than the objects themselves. Two pain points:

- **Heuristic GT $\rightarrow$ anchor assignment**: IoU thresholds (0.7/0.3), one GT $\rightarrow$ multiple anchors – hyperparameters baked into training.
- **Duplicate predictions**: k anchors per location $\times$ overlapping receptive fields $\rightarrow$ many near-duplicate boxes. Requires *NMS* post-processing at inference time (non-differentiable, extra hyperparameters).

2.2.3 Direct Detectors

In order to avoid brittle heuristics we can apply direct detectors that drop anchors + NMS entirely.

Idea: A direct detector outputs a fixed-size set of N predictions $\hat{y} = \{\hat{y}_i\}_{i=1}^N$, where N is chosen larger than the max number of objects in any image. No anchors, no grid; just N slots, each producing $(\hat{p}_i, \hat{b}_i) = (\text{class distribution}, \text{box})$. The ground-truth set y is padded with "no object" $\emptyset$ entries up to size N so both sides have the same cardinality.

Problem: The network outputs N predicted boxes in arbitrary order. In the indirect methods, we used anchor $\rightarrow$ closest GT matching. Here we no longer have these heuristics.

Direct Set Prediction

Via **Bipartite Matching** (NOT training), we can solve the matching as a permutation problem, s.t., σ of predictions that pairs each prediction with exactly one GT (or $\emptyset$) so that the total matching cost is minimized:

$$\hat{\sigma} = \arg \min_{\sigma} \sum_i \mathcal{L}_{\text{match}}(y_i, \hat{y}_{\sigma(i)})$$

$$\text{with } y_i = (c_i, b_i) \quad \hat{y}_{\sigma(i)} = (\hat{p}_{\sigma(i)}, \hat{b}_{\sigma(i)})$$

$$\mathcal{L}_{\text{match}}(y_i, \hat{y}_{\sigma(i)}) = -[c_i \neq \emptyset] \hat{p}_{\sigma(i)}(c_i) + [c_i \neq \emptyset] \mathcal{L}_{\text{box}}(b_i, \hat{b}_{\sigma(i)})$$

combining both the class probability ($\hat{p}_i$) and the box similarity ($\hat{b}_i$) $\rightarrow$ solved in polynomial time using the **Hungarian Algorithm**. Note that we use the non-empty-set instances for both the class prob and the box loss. We do not care about predictions assigned to $\emptyset$, what we want is ONLY the best matches for the GT. With this we do not punish $\emptyset$ -assignments after having found the best GT-matches.

On the best match, we then calculate the **Set Loss**/ Hungarian Loss, where we must tell the network that an unmatched slot should have predicted $\emptyset$ (training), otherwise it would not learn to predict this:

$$\mathcal{L}_{\text{Hungarian}}(y, \hat{y}) = \sum_{i=0}^N -\log \hat{p}_{\hat{\sigma}(i)}(c_i) + [c_i \neq \emptyset] \mathcal{L}_{\text{box}}(b_i, \hat{b}_{\hat{\sigma}(i)}) \quad \text{NLL/CE-loss for strong training signals}$$

Detection Transformer

We can use DETR to *generate direct box predictions*.

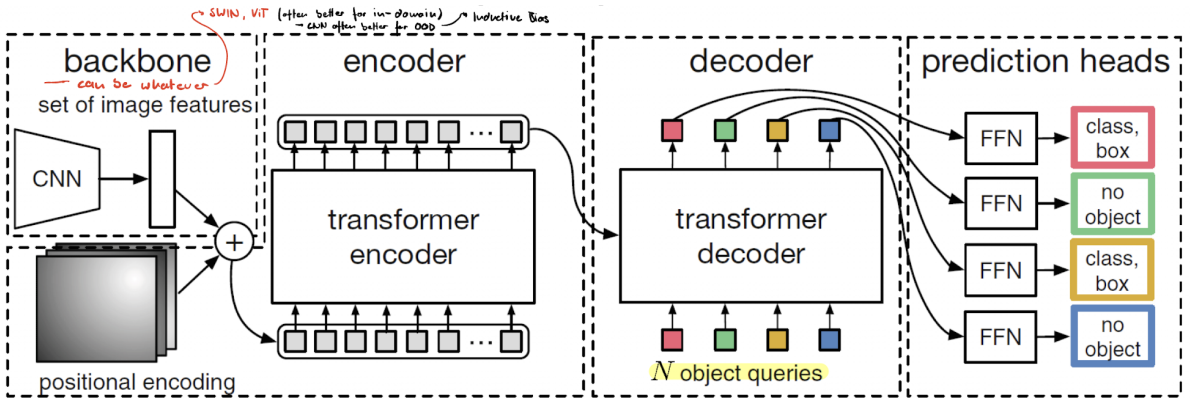


Figure 5: DETR architecture: CNN backbone + positional encoding $\rightarrow$ transformer encoder over flattened image features $\rightarrow$ transformer decoder cross-attending N learned *object queries* against the encoded features $\rightarrow$ shared FFN heads output (class, box) per query, with $\emptyset$ for unused slots.

- **Positional Encoding:** added in the encoder to distinguish same features generated at different spatial locations (transformer is permutation-invariant otherwise).
- **Object Queries:** N *learned* input embeddings added to the input of each attention layer in the decoder $\rightarrow$ each query becomes one prediction slot.
- **Decoder Self-Attention:** lets queries attend to each other $\rightarrow$ pair-wise relations between objects (the mechanism that suppresses duplicates: queries learn to "divide up" the image).
- **Encoder-Decoder Attention** (cross-attention): each query attends over the full encoded feature map $\rightarrow$ whole image as context per object.
- **Prediction Heads:** FFNs with *shared parameters* but *independent computation per query* $\rightarrow (c_i, b_i)$ per slot.

2.3 Instance Segmentation

Definition 10 (Instance Segmentation). *Object Detection with pixel-level masks instead of bounding boxes for objects. Evaluation still via AP.*

2.3.1 Mask R-CNN

We still evolve the R-CNN lineage. The idea is not to *detect-then-segment*.

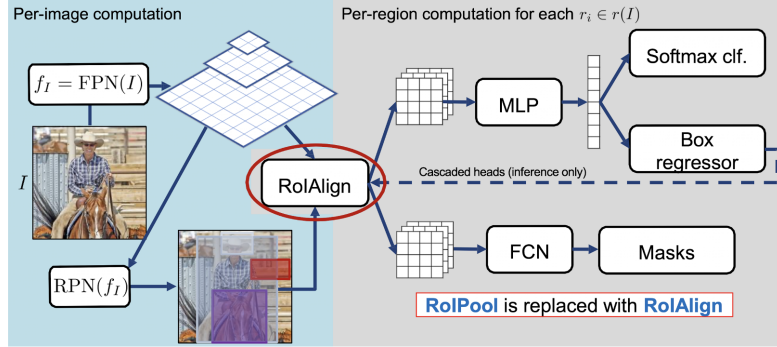


Figure 6: Mask R-CNN: adapted Faster R-CNN with an instance-level segmentation mask head, and RoIAlign.

Core Adaptations of the Mask R-CNN:

- **Instance-Level Segmentation Mask Head:** Decoupled Segmentation from classification such that one mask prediction for each (proposal, class) pair.
- **RoIAlign:** Same concept of transforming an arbitrary-sized proposal into a fixed-dimensional repr, but *without any quantization*.

Why alignment is needed at all: the RPN reads features from integer feature-map cells but writes boxes in continuous image-pixel space (deltas $(dx, dy, dw, dh) \in \mathbb{R}^4$ are continuous on top of the anchor). So when the head re-reads features for that proposal, the box no longer lands on grid cells – that gap is what RoIPool/RoIAlign bridge.

Recall RoIPool snaps twice – (i) project RoI to integer feature-map cells, (ii) snap bin boundaries when subdividing – introducing $\sim$ stride-sized misalignments. Fine for classification, but fatal for per-pixel mask prediction. RoIAlign keeps everything fractional:

1. Project the RoI to the feature map – *do not snap*.
2. Subdivide into a $k \times k$ grid of bins – *do not snap* boundaries.
3. Inside each bin, place a fixed number of *sample points* at exact fractional positions (e.g. uniform $2 \times 2 = 4$ samples per bin).
4. For each sample, get its feature value via **bilinear interpolation** from the 4 nearest feature-map lattice points (features are treated as living at grid *junctions*, not cell centers).
5. Bin output = *average* of the 4 interpolated samples.

Output is still $k \times k \times C$, but spatially faithful and *differentiable* w.r.t. RoI coordinates (RoIPool is not). Key for masks: $14 \times 14 / 28 \times 28$ mask predictions are later resized to the image-space RoI and thresholded at 0.5, so any RoIPool-style quantization error would propagate directly to mask boundaries.

Mask R-CNN Loss

Builds on the Fast R-CNN multi-task loss by adding a per-pixel mask term. For each proposal the mask head outputs a $K \times m \times m$ feature map f (one $m \times m$ mask per class, no softmax across classes), passed through a sigmoid $\rightarrow$ scores s .

- $L_{\text{mask}}(s^u, w) = -\frac{1}{m^2} \sum_{i=1}^{m^2} [w[i] \log s^u[i] + (1 - w[i]) \log(1 - s^u[i])]$ per-pixel *binary* CE on the sigmoid scores of the mask s^u corresponding to the GT class u , against the binary GT mask w . Decouples class from mask – no inter-class competition per pixel (unlike softmax).
- **Combined:** mask loss only on non-background proposals ($[u \geq 1]$):

$$L(p, u, t^u, v, s^u, w) = L_{\text{cls}}(p, u) + [u \geq 1] L_{\text{box}}(t^u, v) + [u \geq 1] L_{\text{mask}}(s^u, w)$$

2.4 Panoptic Segmentation

2.4.1 Fundamentals and Definitions

Definition 11 (Panoptic Segmentation). *For a given, pre-defined set of classes $\mathcal{C}$, assign each pixel in the input image individually to a single class. For each pixel, that belong to a countable class, assign it to a unique instance.*

Panoptic Segmentation unifies the instance and classical semantic segmentation into one task, using a uniform output format and a uniform evaluation metric (**Panoptic Quality**).

Data is in generic supervised learning form: $\mathcal{D} = \{(x_i, y_i) : i = 1, \dots, N\}$. Thus, with Z as the max number of instances for a given class:

Input (RGB): $x_i \triangleq \mathbf{I}_i \in \mathbb{R}^{H \times W \times 3}$

Output: $\mathbf{Y}_i \in \{1, \dots, C\} \times \{1, \dots, Z\}^{H \times W}$

Loss: depending on the architecture (instance-semantic or unified mask-based)

For each pixel i : (class, instance count (if countable)). Contrary to instance segmentation we have **no prediction overlaps** (but arbitrary instance ordering), and **no instance confidence scores**.

Panoptic Quality (PQ).

Fundamental Principle: Each GT segment can have at most one corresponding predicted segment with $IoU > 0.5$ and vice versa. Thus matching is unique and finding it is simply *the first match with $IoU > 0.5$* .

$$PQ = \frac{\sum_{(p,g) \in TP} IoU(p,g)}{|TP| + \frac{1}{2}|FP| + \frac{1}{2}|FN|} \quad \text{for a single class}$$

We can interpret this as a product of segmentation quality (SQ) and recognition quality (RQ):

$$PQ = \underbrace{\frac{\sum_{(p,g) \in TP} IoU(p,g)}{|TP|}}_{SQ} \times \underbrace{\frac{|TP|}{|TP| + \frac{1}{2}|FP| + \frac{1}{2}|FN|}}_{RQ}$$

- **RQ** = F1 score (harmonic mean of precision & recall) – detection quality.
- **SQ** = average IoU over matched (TP) segments – localization/mask quality.
- SQ depends on RQ (only the matched segments enter SQ).
- Decomposition only holds *per class* – does not hold for mean PQ across classes.

2.4.2 Instance-semantic Framework for Panoptic Segmentation

Idea: re-use existing instance and semantic segmenters to obtain a panoptic output without designing a new model → combine their predictions post-hoc.

Baseline: Merge Independent Predictions

Run an instance segmenter (Mask R-CNN, things only) and a semantic segmenter (stuff + things) independently, then merge pixel-wise: things win over stuff; overlapping instances resolved by confidence.

- PQ^{Th} unchanged (instances taken as-is); PQ^{St} drops slightly (instance masks carve into stuff).
- **Failure mode:** *unlabeled (black) halos around boundaries* – semantic seg predicts a thing class but no instance mask covers the pixel → no valid instance ID → pixel unlabeled. Structural flaw of independent models.
- Motivates *backbone-sharing* models (Panoptic FPN).

Panoptic FPN: Shared Backbone

Motivation: the baseline fails structurally because the two segmenters never share features → no way to agree on boundaries (black halos). Sharing a backbone (a) forces the features to be jointly useful for both tasks, (b) is essentially free since FPN already produces *high-resolution, rich, multi-scale* features – exactly the standard requirements for semantic seg.

Starting point: FPN-based Mask R-CNN. Idea: add a *semantic segmentation branch* fed by the same FPN → instance + semantic heads jointly trained on a shared feature pyramid.

- **Semantic branch:** each FPN level (resolutions $1/32, 1/16, 1/8, 1/4$ with 256 channels) is upsampled to a common $1/4$ resolution via a stack of $\text{conv} \rightarrow 2\times \text{upsample}$ blocks (more blocks for coarser levels) producing $128 \times 1/4$ maps, element-wise summed, then $\text{conv} \rightarrow 4\times$ to image resolution $\rightarrow$ per-pixel C -way logits.
- **Training loss:** weighted sum to account for the different ranges of the two losses (instance heads are per-ROI, semantic head is per-pixel over the full image): $L = \lambda_{\text{inst}}(L_{\text{cls}} + L_{\text{box}} + L_{\text{mask}}) + \lambda_{\text{sem}}L_{\text{sem}}$
- **Panoptic inference** (heuristic merge – panoptic format forbids overlap):
 - instance-instance overlap $\rightarrow$ higher confidence wins;
 - instance-stuff overlap $\rightarrow$ instance wins;
 - stuff segments below an area threshold are dropped.
- **Limitation:** heads are still *segregated* – they share a backbone but make independent predictions reconciled by hand-crafted rules $\rightarrow$ suboptimal post-processing. Motivates the unified mask-based formulation below.

2.4.3 Unified Mask-based Framework for Segmentation

Rather than disjointly learning instance and semantic segmentation, we combine them into one unified objective.

Segmentation as Mask-based Classification

Both instance and semantic seg can be written as predicting a *set* of $(\text{mask}, \text{class})$ pairs. This lets us reuse DETR’s set-prediction recipe, only swapping the geometric output from a *box* to a *mask*.

- **Ground truth:** $z^{\text{gt}} = \{(c_i^{\text{gt}}, m_i^{\text{gt}})\}_{i=1}^N$ with $c_i^{\text{gt}} \in \{1, \dots, K\}$, $m_i^{\text{gt}} \in \{0, 1\}^{H \times W}$.
- **Prediction:** $z = \{(p_i, m_i)\}_{i=1}^N$ – N slots, each a class *distribution* p_i and a *soft* mask $m_i \in [0, 1]^{H \times W}$.
- **Matching:** pad z^{gt} with $\emptyset$ tokens up to N , then bipartite-match with the Hungarian algorithm (cf. DETR).

Matching cost per pair (both terms gated on $c_i^{\text{gt}} \neq \emptyset$; raw p – no log – so the two terms stay on the same numerical scale and neither dominates the matching, same logic as DETR):

$$\mathcal{L}_{\text{match}}(z_{\sigma(i)}, z_i^{\text{gt}}) = -[c_i^{\text{gt}} \neq \emptyset] p_{\sigma(i)}(c_i^{\text{gt}}) + [c_i^{\text{gt}} \neq \emptyset] \mathcal{L}_{\text{mask}}(m_{\sigma(i)}, m_i^{\text{gt}})$$

$\mathcal{L}_{\text{mask}}$: binary mask overlap loss (details later).

Hungarian / set loss (a.k.a. *mask-classification loss*) given the optimal match $\hat{\sigma}$:

$$\mathcal{L}_{\text{mask-cls}}(z, z^{\text{gt}}) = \sum_{i=1}^N -\log p_{\hat{\sigma}(i)}(c_i^{\text{gt}}) + [c_i^{\text{gt}} \neq \emptyset] \mathcal{L}_{\text{mask}}(m_{\hat{\sigma}(i)}, m_i^{\text{gt}})$$

Same DETR-style asymmetry: the cls term is *not* gated by $[c_i^{\text{gt}} \neq \emptyset]$ – unmatched slots are explicitly supervised toward the $\emptyset$ class to suppress duplicates; switching to $-\log p$ gives proper NLL gradients during training. Mask term stays gated (no GT mask for $\emptyset$ slots).

MaskFormer

Similar idea as DETR but *without a transformer encoder* – segmentation needs per-pixel features at near-image resolution, and self-attention is $\mathcal{O}(N^2)$ in tokens, so flattening high-res feature maps into an encoder would be prohibitive. Instead a *pixel decoder* (FPN-style CNN upsampling) produces the dense per-pixel features that the transformer decoder’s queries cross-attend against.

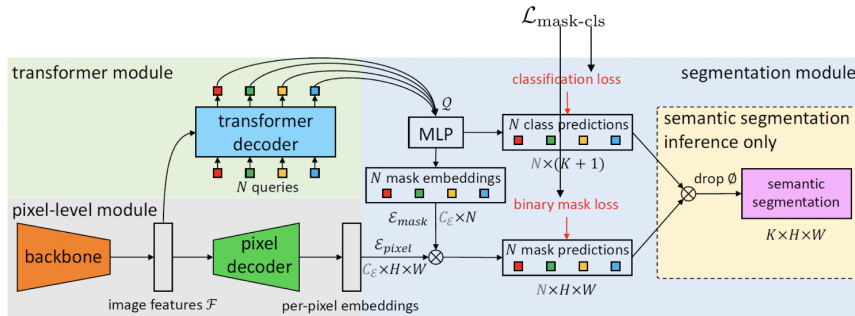


Figure 7: MaskFormer High Level View

Mask Prediction

Sigmoid dot product of per-query *mask embedding* $\mathcal{E}_{\text{mask}}[:, i]$ with per-pixel embedding $\mathcal{E}_{\text{pixel}}[:, h, w]$ from the pixel decoder: $m_i[h, w] = \sigma(\mathcal{E}_{\text{mask}}[:, i]^T \cdot \mathcal{E}_{\text{pixel}}[:, h, w])$ Interpreted as the *non-exclusive degree* to which pixel (h, w) belongs to mask i – no softmax across slots, so different slots’ masks can overlap. The panoptic no-overlap constraint is enforced only at inference (cf. Mask R-CNN’s per-class masks, here per-query).

Same Architecture, Different Weights

One architecture for semantic / instance / panoptic, but a *separate model is trained per task* – unified *formulation*, not unified *weights* (Mask2Former goes further).

Inference

Training is fixed (mask-cls loss); only the inference rule switches per task:

- **Panoptic / instance:** per pixel, pick the *dominant* (slot, class) pair, $\arg \max_{i: c_i \neq \emptyset} p_i(c_i) \cdot m_i[h, w]$ – both class- and mask-conf must be high. Each instance stays a distinct segment.
- **Semantic:** *marginalize* over slots, $\arg \max_{c \in \{1, \dots, K\}} \sum_{i=1}^N p_i(c) \cdot m_i[h, w]$. Semantic only cares about the class per pixel, not which slot produced it $\rightarrow$ multiple ”person” slots can pool their evidence into one ”person” region.
- Training the semantic objective directly would scatter the gradient across all N queries (each gets only a fraction of the signal) $\rightarrow$ keep mask-cls loss, switch inference only.
- However, specialized methods require significantly fewer epochs for equivalent performance

Mask2Former

Same fundamental idea as MaskFormer but with significant improvements:

- **Masked Attention** (in cross-attention): restrict each query to attend *only to pixels inside its current mask prediction*, by adding an attention bias $\mathcal{M}_{l-1}(x, y) = \begin{cases} 0 & \text{if } M_{l-1}(x, y) = 1 \\ -\infty & \text{otherwise} \end{cases}$ inside the softmax: $X_l = \text{softmax}(\mathcal{M}_{l-1} + Q_l K_l^T) V_l + X_{l-1}$. $-\infty$ entries vanish after softmax $\rightarrow$ those pixels contribute zero. Intuition: *global context takes time to learn*; for cross-attention local foreground features already suffice, broader context is gathered by self-attention.
- **Swap Self- and Cross-Attention:** queries at the first decoder layer are image-independent (purely learned embeddings) $\rightarrow$ self-attention among them carries no image content to share. Do cross-attention first to inject image features into queries, *then* self-attention – opposite of the transformer default.
- **High-Res Features from Pixel Decoder:** MaskFormer decoded queries against the *low-res bottleneck* features only; Mask2Former feeds each successive transformer-decoder layer a *different scale* from the pixel decoder (multi-scale). Helps segment objects of diverse scales, especially small ones.
- **Point Sampling** (matching cost & mask-cls loss approximation): $\mathcal{L}_{\text{mask}}$ over the full $H \times W$ map for every (prediction, GT) pair is memory-heavy. Approximate it on K sampled pixels:
 - *Matching cost:* *uniform* sampling of a *shared* pixel set across all pairs $\rightarrow$ costs stay comparable across pairs (apples-to-apples for the Hungarian).
 - *Mask-cls loss:* *importance* sampling of *distinct* pixel sets per mask $\rightarrow$ each mask gets its own boundary/hard-pixel focus.

Drops the full-image mask compute requirement $\rightarrow$ enables training on standard hardware.

3 Part III: Geometric Perception and Localization

3.1 Depth and 3D Estimation

Stereo vision to generate 3D renderings from 2D images. Other possibilities include: *structured light*, *ultrasound*, *radar*, *time of flight (active)*, and *cam/lidar combinations*. We can deduce some information also from 2D images using cues including shading, texture, focus, and motion.

Challenges to monocular depth estimation: a single 2D image may be produced from an infinite number of 3D scenes due to **scale ambiguity** → overcome some of this by using cues or features such as perspective or texture information, object sizes, locations and occlusions.

→ regression problem (cont output per pixel), thus we use MSE or variants. (no longer MAP!)

Ground Truth.

- **LiDAR** (e.g. Velodyne): sparse; limited vertical FoV; errors on moving objects and on specular/transparent surfaces. In practice can be *densified* for static parts by merging consecutive frames (imperfect, but less sparse).
- **Structured light** (e.g. Kinect): fails outdoors (sunlight interference); short range (~few meters); errors on specular/transparent surfaces.

Evaluation Metrics

Comparing predicted depth map Y to GT Y^* over T valid depth pixels.

- **Threshold accuracy**: fraction of pixels with $\max\left(\frac{y_i}{y_i^*}, \frac{y_i^*}{y_i}\right) < \text{thr}$ (symmetric ratio ≥ 1 → penalizes over- and under-estimation). Counts correct pixel given some tolerance; typical threshold 1.25. Robustness-flavored.
- **Absolute relative difference**: $\text{rel} = \frac{1}{T} \sum_{i,j} |y_{i,j} - y_{i,j}^*| / y_{i,j}^*$ – per-pixel error normalized by GT depth (far pixels not over-weighted).
- **Squared relative difference**: $\text{srel} = \frac{1}{T} \sum_{i,j} |y_{i,j} - y_{i,j}^*|^2 / y_{i,j}^*$ – penalizes larger errors more.
- **RMS (linear)**: $\sqrt{\frac{1}{T} \sum_{i,j} |y_{i,j} - y_{i,j}^*|^2}$ – standard L2; dominated by far/large-depth errors.
- **RMS (log)**: $\sqrt{\frac{1}{T} \sum_{i,j} |\log y_{i,j} - \log y_{i,j}^*|^2}$ – compares depth *magnitudes* (orders of magnitude); equalizes near/far contributions.

3.1.1 Supervised Methods

Standard CNN based Enc-Dec methods with loss between output and GT depth.

Loss: While $L1$ and $L2$ are possible, the **berHu loss** is more common: $\mathcal{B}(x) = \begin{cases} |x| & |x| \leq c \\ \frac{x^2 + c^2}{2c} & |x| > c \end{cases}$

Inverse of Huber: L1 for small residuals, L2 for large ones.

Computed as $\mathcal{B}(\tilde{y} - y)$ with $c = \frac{1}{5} \max_i (|\tilde{y}_i - y_i|)$ over all pixels in the current batch – the L1/L2 cut-off is set to 20% of the largest residual in the batch (adaptive scale).

P3Depth.

Leveraging regularities in real-world (planar) scenes yielding piecewise-smooth depth maps. → rather than directly predicting depth D , predict a per-pixel repr $\mathbf{C}$ that is *identical for pixels on the same 3D plane*, then use neighbor information in $\mathbf{C}$ to predict the depth of each pixel.

Plane Coefficient Representation

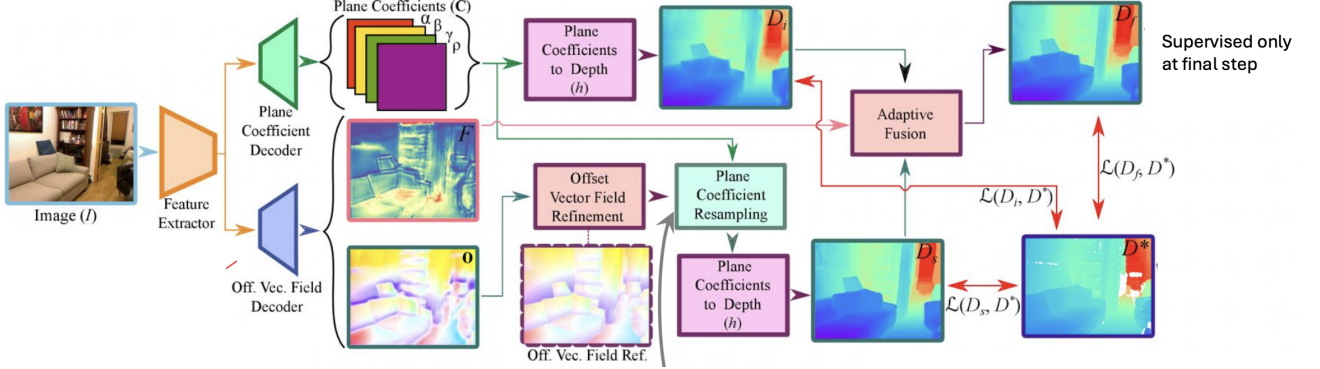
Backproject a pixel $\mathbf{p} = (u, v)^\top$ to a 3D point $\mathbf{P} = (X, Y, Z)^\top$ via the pinhole model: $Z = D(u, v)$, $X = \frac{Z(u - u_0)}{f_x}$, $Y = \frac{Z(v - v_0)}{f_y}$. A 3D plane with normal $\mathbf{n} = (a, b, c)^\top$ and offset d satisfies $\mathbf{n} \cdot \mathbf{P} + d = 0$. Substituting the backprojection and rearranging gives a *linear function for $1/Z$ in pixel coords*:

$$\frac{1}{Z} = \underbrace{\frac{-a}{f_x d}}_{\hat{\alpha}} u + \underbrace{\frac{-b}{f_y d}}_{\hat{\beta}} v + \underbrace{\frac{1}{d} \left(\frac{a}{f_x} u_0 + \frac{b}{f_y} v_0 - c \right)}_{\hat{\gamma}} \quad \text{Normalize: } \rho = \sqrt{\hat{\alpha}^2 + \hat{\beta}^2 + \hat{\gamma}^2} \quad (\alpha, \beta, \gamma) = (\hat{\alpha}, \hat{\beta}, \hat{\gamma}) / \rho$$

→ **plane coefficients** $\mathbf{C} = (\alpha, \beta, \gamma, \rho)^\top$. Depth recovered from $\mathbf{C}$ and pixel coords: $Z = [(\alpha u + \beta v + \gamma)\rho]^{-1} = h(\mathbf{C}, u, v)$. Pixels on the same 3D plane share the same $\mathbf{C}$ – it depends only on camera intrinsics and the plane, not the individual pixel → enables neighborhood-based depth refinement.

Injecting Regularities

In order to actually use these **interaction between pixels in the new output space**, we must fuse the spaces. Simply changing the output space to a repr more closely related to 3D planes does NOT inject these priors.



Two decoders branch off a shared feature extractor:

- **Plane Coefficient Decoder** → dense plane-coefficient map $\mathbf{C}(u, v) = (\alpha, \beta, \gamma, \rho)$.
- **Offset Vector Field Decoder** → per-pixel offset $\mathbf{o}(\mathbf{p}) = \mathbf{q} - \mathbf{p}$ pointing to a **seed pixel** $\mathbf{q}$ in the neighborhood that is associated with the *same 3D plane* as $\mathbf{p}$.

Two depth predictions are then computed:

- **Initial:** $D_i(u, v) = h(\mathbf{C}(u, v), u, v)$ – each pixel uses its own coefficients.
- **Seed-based:** resample coefficients at the seed location, $\mathbf{C}_s(\mathbf{p}) = \mathbf{C}(\mathbf{p} + \mathbf{o}(\mathbf{p}))$, then $D_s(u, v) = h(\mathbf{C}_s(u, v), u, v)$ – uses the seed’s plane params but $\mathbf{p}$ ’s pixel coords (valid since both share the same plane).

Why the seed at all? CNN/transformer decoders produce smooth feature maps → plane coefficients are reliable in plane *interiors* but blur near plane boundaries. The offset snaps each pixel to a more reliable interior seed of its own plane → sharp depth discontinuities at edges.

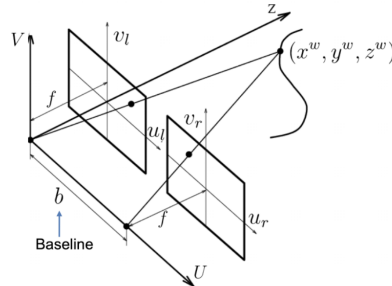
Adaptive Fusion: $D_f(u, v) = F(u, v)D_s(u, v) + (1 - F(u, v))D_i(u, v)$, with predicted confidence $F \in [0, 1]$. High F trusts the piecewise-planar prior; low F falls back to direct prediction for non-planar surfaces (curved, high-texture).

Training: supervised *only on depth*: $\mathcal{L}(D_i, D^*) + \mathcal{L}(D_s, D^*) + \mathcal{L}(D_f, D^*)$. Offsets, plane coefficients, and confidence are all learned *implicitly* – no GT for them.

3.1.2 Stereo Depth Estimation

Using a stereo setup we can estimate depth using triangulation.

We consider *identical cameras, with coplanar image planes, aligned x-axes and the convention of setting the left camera frame to the world coords*. Thus have a translation along the known baseline (in meters) and no rotation.



The translation from the $C_L = W$ to C_R is then given by:

$$\begin{pmatrix} x^r \\ y^r \\ z^r \\ 1 \end{pmatrix} = \begin{pmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{pmatrix} \begin{pmatrix} x^w \\ y^w \\ z^w \\ 1 \end{pmatrix} \quad \text{with} \quad \mathbf{R} = \mathbf{I} \text{ and } \mathbf{C} = (b \ 0 \ 0)^T = -\mathbf{t}$$

Projection & Disparity

Further assume image coords already centered at the principal point $((u_0, v_0) = (0, 0))$ and $K_L = K_R = K$ with $K = \text{diag}(kf, lf, 1)$. Projecting the world point through each camera: $\rho \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = K \begin{pmatrix} x^w \\ y^w \\ z^w \end{pmatrix}$, $\rho' \begin{pmatrix} u' \\ v' \\ 1 \end{pmatrix} = K \begin{pmatrix} x^w - b \\ y^w \\ z^w \end{pmatrix}$

Dehomogenizing:

$$u = \frac{kfx^w}{z^w}, v = \frac{lfy^w}{z^w}, \quad u' = \frac{kf(x^w - b)}{z^w}, v' = \frac{lfy^w}{z^w}$$

$\rightarrow v = v'$: rectified pair, the same scene point lies on the same image row in both cameras (only horizontal disparity).

Disparity: $d = u - u'$ – the horizontal displacement of the same point between the two images. Solving the equations above for the 3D coords:

$$x^w = b \frac{u}{u - u'}, \quad y^w = b \frac{k}{l} \frac{v}{u - u'}, \quad z^w = bk \frac{f}{u - u'}$$

Depth is inversely proportional to disparity. Normally $k = l$ (square pixels). Near objects $\rightarrow$ large d (well-resolved); far objects $\rightarrow$ small d (precision degrades quadratically with distance).

Visible Region

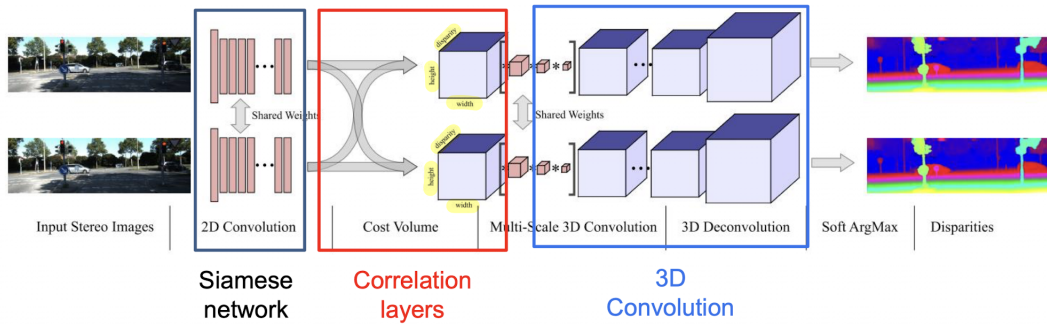
- **Blind region:** zone in front of the cameras visible to only one (or neither) $\rightarrow$ no stereo depth possible.
- **Valid stereo region:** intersection of both FoVs $\rightarrow$ correspondences (hence depth) computable.

Design Trade-offs & Remarks

- Increasing b and/or $f \rightarrow$ better depth resolution, but smaller simultaneous visibility (larger blind region; narrower FoV for f).
- *Same disparity = same depth* holds *only for this rectified simple setup* – not for general stereo geometries.
- Too-small disparity $\rightarrow$ depth becomes unresolvable: sub-pixel matching noise translates to large depth ambiguities (quadratic in distance) $\rightarrow$ far objects unreliable.
- The real bottleneck is **finding correspondences** under non-ideal inputs: occlusions, repetitive patterns, textureless regions, reflective surfaces.

Stereo Matching Algorithm and Siamese Networks.

Classical four-step process: (I) matching cost, (II) cost aggregation, (III) optimization, (IV) disparity refinement. **Siamese Networks** = two twin networks with *shared parameters* processing two inputs in parallel, trained to minimize the distance for matching pairs and maximize it for distinct ones $\rightarrow$ natural fit for step (I).



End-to-end deep stereo regression replaces all four steps with learned blocks:

- **Siamese 2D conv** – shared-weight feature extractor on both images $\rightarrow$ learned per-pixel descriptors (replaces hand-crafted matching cost of step I).
- **Cost volume** (a.k.a. *correlation layer*) – *no learnable parameters*; a fixed construction that for every (u, v) and candidate disparity d combines $f_L(u, v)$ with $f_R(u - d, v)$ – by concatenation $(H \times W \times (D_{\max} + 1) \times 2C)$ or dot product $(H \times W \times (D_{\max} + 1))$. Built in *both directions* (L $\rightarrow$ R and R $\rightarrow$ L) $\rightarrow$ two cost volumes, one anchored per image, enabling downstream left-right consistency / occlusion reasoning. Still differentiable $\rightarrow$ gradients flow back into the Siamese features. It is the *inductive bias* of the architecture (only same-row, fixed-shift pairs are considered).

- **Multi-scale 3D conv / 3D deconv** – propagates context across spatial *and* disparity dimensions → cost aggregation + optimization (steps II+III). *Why 3D conv, not 2D?* 2D conv would slice across (H, W) per disparity independently, decimating the disparity axis; 3D conv treats D as a proper spatial dim and preserves it. Geometry-motivated design, not a black box. Output: a refined cost volume reduced to a *single feature channel* – $H \times W \times (D_{\max} + 1)$ scalar costs.
- **Soft argmax / soft argmin** – differentiable disparity selection (step III). A hard argmin over costs would (i) only allow integer disparities → quantized depth and (ii) make the loss *fragile*: a prediction one off from GT incurs the same loss as one completely off (no gradient pointing toward correct). Instead treat costs as negated logits, softmax over the disparity axis, take the convex combination: $\hat{d}[i, j] = \sum_{D=0}^{D_{\max}} D \cdot \text{softmax}_D(-C[i, j, D])$ → continuous, sub-pixel disparity that flows gradients smoothly. Final loss: L_1 regression

$$L = \frac{1}{HW} \sum_{i,j} \|\hat{d}[i, j] - d[i, j]\|_1$$

Why a cost volume rather than a naive concat? A naive baseline would just concatenate the left and right feature maps along channels and regress disparity with a big CNN – but this throws away the stereo geometry, forcing the network to *discover* that matches lie along the same row at a small horizontal shift. The cost volume bakes that prior in: each disparity slice d holds the similarity between the left feature at (u, v) and the right feature at $(u - d, v)$ → only same-row, fixed-shift correspondences are considered, so the network just has to *pick* the best disparity.

3.1.3 Self-Supervised Monocular Depth Estimation from Stereo

Key asymmetry: stereo is used at *training* time as a supervision signal; the CNN only ever sees the *left* image.

- **Forward pass:** $\text{CNN}(L) \rightarrow$ disparity map.
- **Loss:** warp R using the predicted disparity (via the stereo projection equations $u = \frac{kf x^w}{z^w}, u' = \frac{kf(x^w - b)}{z^w}$) → synthesized $\tilde{L}$; photometric loss $\|\tilde{L} - L\|$. Right image is supervision only, not input.
- **Why "monocular":** at test time only one camera is needed. Stereo provides cheap, scalable self-supervision
- **Accuracy:** not as good as fully supervised, but decent.
- **Issues:** occlusions, FoV mismatch between cameras (some pixels in L have no valid match in R).

3.1.4 Self-Supervised Depth and Motion Estimation from Video

Extends stereo self-supervision to *monocular video*: no known baseline, so the camera motion between consecutive frames must also be predicted. Train on raw video, deploy on a single image.

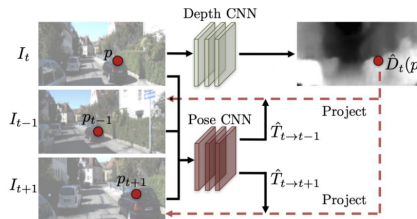


Figure 8: Unsupervised Learning of Depth and Ego-Motion from Video

Two jointly trained networks:

- **Depth CNN:** $I_t \rightarrow \hat{D}_t(p)$ – per-pixel depth of the target frame.
- **Pose CNN:** $(I_t, I_{t\pm 1}) \rightarrow \hat{T}_{t \rightarrow s}$ – relative camera pose (*ego-motion*) between target and source frames.

Warping equation: for a pixel p_t in the target, the corresponding source pixel is $p_s \sim K \hat{T}_{t \rightarrow s} \hat{D}_t(p_t) K^{-1} p_t$ (unproject p_t via K^{-1} , scale by depth, transform by pose, reproject via K). The source image is then sampled at p_s via *bilinear interpolation* (differentiable w.r.t. warp → gradients reach both networks).

View-synthesis photoconsistency loss: $\mathcal{L}_{vs} = \sum_s \sum_p |I_t(p) - \hat{I}_s(p)|$ – L1 between target and warped source frame(s). Depth and pose are supervised *only* through this signal.

Failure modes: breaks on specular / non-Lambertian surfaces (photometric assumption violated); moving objects violate the static-scene assumption → commonly worked around by global motion params or learned masks.

3.2 3D Classification, Segmentation, and Detection

3.2.1 Foundations and Definitions

Representations: many possibilities including multi-viewm volumetric (voxels), part assembly, **Point Clouds**, mesh (graph CNN), and implicip shapes (functional repr.) → focus on point clouds since close to raw sensor data and common standard. Point Clouds can either be converted into other reprs to feed into NNs or we can do *feature learning* directly on point clouds (our focus)

3D Perception Tasks: Classification, Segmentation, Segmentation, Detection, Correspondences, Reconstruction

3.2.2 3D Classification and Semantic Segmentation on Point Clouds

PointNet.

Point Cloud based end-to-end learning for *scattered, unordered, sparse, noisy* point data for various tasks. Challenges caused by the data foundation:

- **Unordered** input point set: Model requires invariance to $N!$ permutations (N orderless points). This can be achieved via *Symmetric Functions* (c.f. GNNs): Trivial Solution is the *MLP with max pooling* (across features):

$$\underbrace{x}_{\text{input}} \rightarrow \underbrace{\text{MLP}(x_i)}_h \rightarrow \underbrace{\text{max_pool}}_g \rightarrow \underbrace{\text{MLP}}_\gamma \quad \gamma \text{ ensures permutation invariance}$$
- **Geometric Transformation Invariance:** rotations / Euclidean transformations must not alter results. Solution: **T-Net** – a mini vanilla PointNet whose output is a transformation *matrix* (not a class score), applied by matrix-mult to align inputs to a *canonical space*. Used twice: 3×3 on raw $n \times 3$ points (*input* T-Net), 64×64 on intermediate $n \times 64$ features (*feature* T-Net). Data-dependent → no hard-coded rotations.

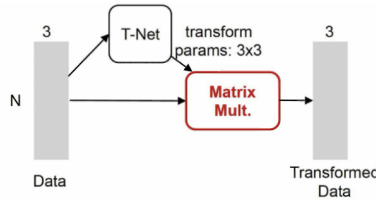


Figure 9: T-Net for Input Alignment by Transformer Network

PointNet Classification Network

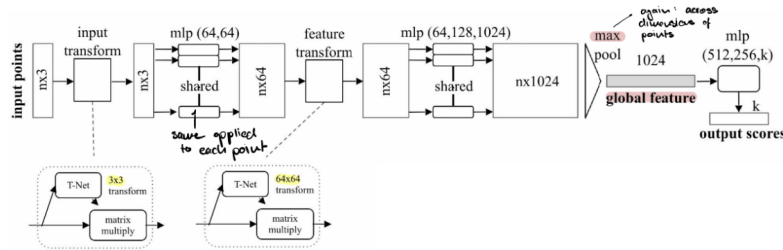


Figure 10: PointNet Classification Network

Whole-cloud → one class. Input: unordered $n \times 3$ points; output: k class scores. Invariances required: permutation (handled by per-point shared MLPs = $h + \text{max-pool} = g$) and rigid transformation (two T-Nets); final MLP = γ .

PointNet Segmentation Network

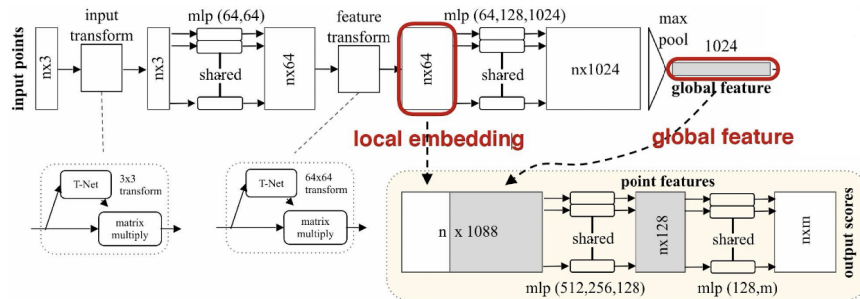


Figure 11: PointNet Segmentation Network

The local embeddings encode point specific details and are non-hierarchical. To each such local embedding we add (concat) the global features so each point sees scene-level context.

PointNet Limitations

- **No local context per point:** features are either *per-point* (one point seen at a time) or *global* (all points aggregated by max-pool) – nothing in between. Contrast with 3D CNNs that have hierarchical feature learning across multiple receptive-field scales.
- **Global feature depends on absolute coordinates:** MLPs ingest raw (x, y, z) , so the global descriptor is tied to absolute position $\rightarrow$ same object at a different location yields a different feature $\rightarrow$ hard to generalize to unseen scene configurations.

PointNet++

Addresses local context per point and global feature dependence on absolute coordinates via hierarchical local-neighborhood processing. The goal is to *mimic convolution* style neighborhood processing.

In scattered point clouds, however, we cannot apply simple convolution, thus we use one of the following to find neighbors:

- **Ball Query:** fix a radius r , take all points within. Fixed metric "kernel" $\rightarrow$ more stable across density variations (the receptive field is the same scale everywhere). Variable number of neighbors.
- **K-NN Query:** distance problematic, since K can represent very distant context if static.

Set Abstraction Levels

Hierarchical structure = a stack of *set abstraction* levels, each repeating three steps:

1. **Sampling Layer:** *Iterative Farthest Point Sampling* (FPS) – given the current sampled set S , the next point is $x_{\text{next}} = \arg \max_{x \notin S} \min_{s \in S} \|x - s\|$ (the unsampled point whose *nearest sampled neighbor* is farthest) $\rightarrow$ each new point fills the largest gap $\rightarrow$ *better coverage* than random sampling.
2. **Grouping Layer:** For each centroid, run ball query (or K-NN) $\rightarrow$ output groups $N' \times K \times (d + C)$. K varies per group (ball query is radius-based); the next PointNet collapses the variable- K axis to a fixed-length feature via max-pool.
3. **PointNet Layer:** Apply PointNet inside each group, taking points *relative* to the centroid $\rightarrow$ one feature per centroid (the analog of one convolution output).

Stacking: N points $(x, y) \rightarrow N_1$ anchors $(x, y, \mathbf{f}) \rightarrow N_2$ anchors $(x, y, \mathbf{f}')$ $\rightarrow \dots$ – fewer points, richer features per level (CNN-style downsampling). Two limitations of vanilla PointNet are fixed: (i) each anchor's feature now *sees its local neighborhood* $\rightarrow$ hierarchical local context; (ii) since inner PointNets see *relative* coordinates, the features are translation-invariant.

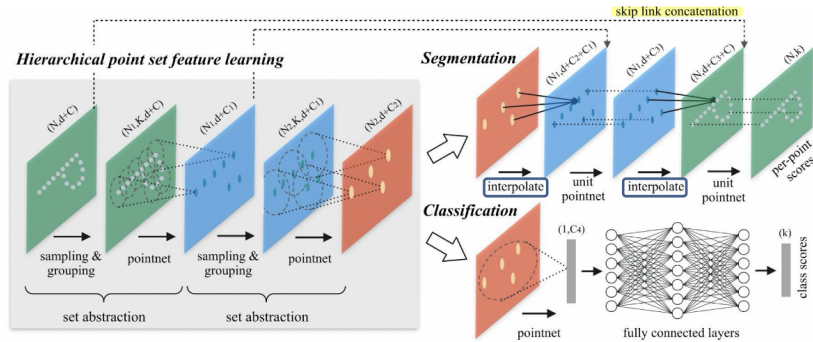


Figure 12: PointNet++ Architecture

Feature Interpolation for PointNet++ Decoder: For per-point tasks (segmentation) we need to undo the set-abstraction subsampling and propagate features back to denser resolution. For each output point $\mathbf{x}_j$, find its k nearest neighbors $\{\mathbf{x}_i\}$ in the (subsampled) input set, then compute an inverse-distance-weighted average:

$$w_{ij} = \frac{1}{d(\mathbf{x}_j, \mathbf{x}_i)^p}, \quad \mathbf{f}(\mathbf{x}_j) = \frac{\sum_{i=1}^k w_{ij} \mathbf{f}(\mathbf{x}_i)}{\sum_{i=1}^k w_{ij}}$$

Closer subsampled points contribute more; typically $k = 3, p = 2$. Density-aware upsampling – the decoder analog of bilinear upsampling in 2D CNNs.

3.2.3 3D Object Detection on Point Clouds

Target: 3D BB around each object given *3 DOF for size*, *3 DOF for center*, and *1 DOF for rotation* - assuming object on ground (no vertical rotation).

LiDAR provides reliable depth information used to localize and characterize objects. However, LiDAR point clouds are *sparse, irregular and have highly variable point density*. We can address these challenges by *manually crafting feature repr* for point clouds, *point cloud projection into perspective view* for image-based techniques, or *rasterize point clouds into 3D voxel grids* and encode each voxel with hand-crafted or PointNet features.

VoxelNet

While a pure 2D inspired sliding window and classification approach is theoretically possible, the search space is huge (cubic increase with resolution).

→ two-stage approach **object proposal + refinement**

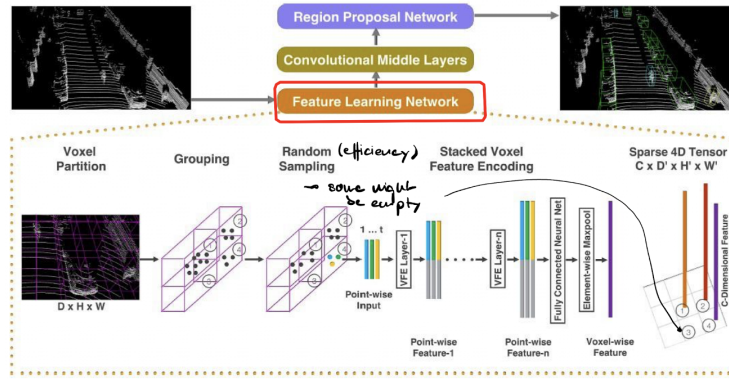


Figure 13: VoxelNet Architecture with Feature Learning Network Details

- **Feature Learning Network:** turns a raw $\sim 100K$ -point LiDAR cloud into a dense voxel-wise feature grid.
 1. **Voxel Partition:** evenly partition 3D space into a $D \times H \times W$ voxel grid.
 2. **Grouping:** assign each point to its voxel; densities are highly uneven.
 3. **Random Sampling:** cap each voxel at T points → (i) compute savings, (ii) reduces sampling bias from per-voxel point-count imbalance, adds training variation.
 4. **Stacked Voxel Feature Encoding (VFE)** – per non-empty voxel $V = \{\mathbf{p}_i = [x_i, y_i, z_i, r_i]^\top\}_{i=1..t}$ (r_i = LiDAR return intensity):
 - Augment each point with its offset to voxel centroid: $\hat{\mathbf{p}}_i = [x_i, y_i, z_i, r_i, x_i - v_x, y_i - v_y, z_i - v_z]^\top \in \mathbb{R}^7$.
 - Shared MLP (linear + BN + ReLU) → per-point features → max-pool across the voxel for a voxel-level feature → concat back per point (PointNet-seg local+global).
 - Stack VFE layers; final = one feature per non-empty voxel (→ sparse 4D tensor).
- **Convolutional Middle Layers:** stack of $3D \text{ conv} + BN + ReLU$ blocks on the sparse voxel tensor. Progressively expanding receptive field aggregates voxel-wise features into richer shape descriptions.
Key step: collapse the vertical (z) dimension to a singleton → output is a 2D **BEV (Bird's-Eye-View) feature map**. Driving is essentially planar, so dropping height makes downstream 2D detection feasible.
- **Region Proposal Network:** standard 2D Faster-RCNN-style RPN, now applicable because the middle layers already produced a BEV map. Two specializations for driving: (i) anchors only at *two* orientations (longitudinal + frontoparallel) since vehicles in traffic appear in essentially two poses, (ii) regression outputs **7-DoF 3D boxes** $(x, y, z, w, l, h, \theta)$ instead of 4-DoF 2D ones – 2D detection geometry, 3D output.

Bounding Box Regression

GT box parameterized as $(x_c^g, y_c^g, z_c^g, l^g, w^g, h^g, \theta^g)$ (center, length/width/height, yaw around z); anchor analogously with superscript a . Residual target $\mathbf{u}^* \in \mathbb{R}^7$ encoding the GT box *relative* to the anchor:

$$\Delta x = \frac{x_c^g - x_c^a}{d^a}, \quad \Delta y = \frac{y_c^g - y_c^a}{d^a}, \quad \Delta z = \frac{z_c^g - z_c^a}{h^a}, \quad \Delta l = \log \frac{l^g}{l^a}, \quad \Delta w = \log \frac{w^g}{w^a}, \quad \Delta h = \log \frac{h^g}{h^a}, \quad \Delta \theta = \theta^g - \theta^a$$

with the anchor’s BEV diagonal $d^a = \sqrt{(l^a)^2 + (w^a)^2}$. Normalizing BEV centers by d^a (and height offset by h^a) makes the regression *scale-invariant* in anchor size; *log-ratios* for dimensions turn multiplicative size errors into additive ones and guarantee positivity after exp. Direct analog of Faster R-CNN’s 2D box parameterization, extended to 3D + rotation.

Loss

Multi-task loss, Faster-R-CNN style:

$$L = \alpha \frac{1}{N_{\text{pos}}} \sum_i L_{\text{cls}}(p_i^{\text{pos}}, 1) + \beta \frac{1}{N_{\text{neg}}} \sum_j L_{\text{cls}}(p_j^{\text{neg}}, 0) + \frac{1}{N_{\text{pos}}} \sum_i L_{\text{reg}}(\mathbf{u}_i, \mathbf{u}_i^*)$$

- **Classification:** cross-entropy on both *positive* and *negative* anchors (separately normalized), weighted by α, β to balance their relative contribution.
- **Regression:** smooth- L_1 on the 7-D residual $\mathbf{u}_i$, *gated to positive anchors only* – negatives have no GT box to regress against (same gating as Faster R-CNN RPN).

Note: Detection range and per-voxel cap T are tuned *per class*: cars use a wider range (~ 70 m) with $T = 35$; pedestrians/cyclists a shorter range (~ 48 m) with $T = 45$ – smaller targets are denser per voxel but only relevant nearby.

Why operate in 3D? Native 3D operations factor out pose variations and let the model directly exploit geometric structure (planarity, symmetry, repetition) that 2D projections obscure – e.g. the VFE’s centroid-relative offsets are a 3D-native prior.

PointRCNN

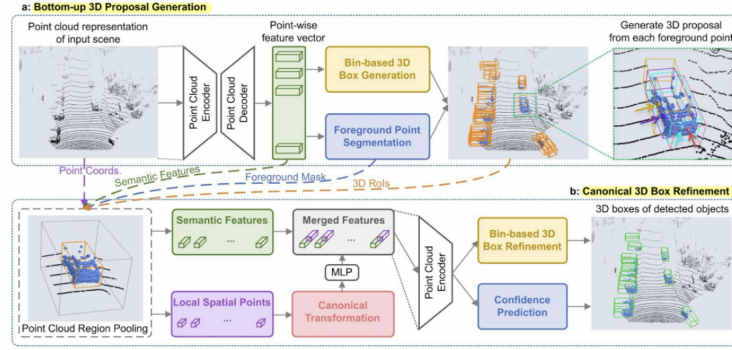


Figure 14: PointRCNN Architecture

- **Bin-based 3D Box Generation:** per foreground point p , instead of regressing the box center as one continuous offset, *discretize the location space into bins* and use a hybrid **classification + residual regression** head.
 - Around p , a square location space of half-side $\mathcal{S}$ is split into $\mathcal{C}$ bins per dimension (bin width $\delta = 2\mathcal{S}/\mathcal{C}$). For $u \in \{x, y\}$: $\text{bin}_u^{(p)} = \lfloor (u^p - u^{(p)} + \mathcal{S})/\delta \rfloor$ (GT-center bin index), with residual $\text{res}_u^{(p)}$ measuring the offset within the bin center. Same for yaw θ .
 - Other dimensions (z, h, w, l) are regressed directly (no binning – height range is small, sizes are smooth).
 - **Hybrid loss:** cross-entropy on bin indices (coarse localization) + smooth- L_1 on residuals (fine offset), averaged over positive proposals. *No objectness loss*: proposals come from foreground points, so they’re always positive by construction.
 - *Why*: classification is more *robust for coarse localization* than direct regression – splitting into coarse-class + fine-residual stabilizes training, especially for far/sparse points.
- **Canonical Transformation** (stage-2 refinement): for each proposal, transform its pooled points into a *box-local* Canonical Coordinate System (CCS) – origin at the proposal center, axes aligned with its orientation. Each proposal gets its own CCS.
 - *Why*: removes the proposal’s global pose dependency $\rightarrow$ all boxes look alike in their own frame $\rightarrow$ refinement network learns local geometry consistently across proposals.
 - *What is transformed vs kept global*: only the raw spatial coordinates of pooled points are canonicalized; the *semantic/high-level features* from stage 1 stay in the original frame (they encode global scene context).
 - Canonical spatial features + global semantic features are then *merged* (concat $\rightarrow$ MLP, then PointNet++ encoder), feeding two refinement heads: a second bin-based 3D box head + a confidence prediction.

3.3 3D Reconstruction and Localization

4 Part IV: Robust Perception: Multi-Modal, Multi-Domain and Multi-Agent Methods

5 Part V: Temporal Perception